TECHNICAL REFERENCE · DESIGN & VERIFICATION

4×4 Weight-Stationary Systolic Accelerator

Complete architecture, RTL structure and cycle-level data flow of a 13-module SystemVerilog matrix-multiply engine — with cycle-accurate animations of the delay pipes, the skew buffers and the systolic array itself.

`13 RTL modules` `13 testbenches` `13/13 PASS · 3 902 checks · 0 mismatches`
`int8 × int8 → int32` `QuestaSim 2021.1` `2026-07-30`

READING THIS AS A PDF

Every animated figure in the HTML edition is followed here by its **complete cycle-by-cycle trace table**, generated by the same cycle-accurate model that drives the animation. Nothing is lost — the tables contain every value the animation would show, for every clock edge. Open `architecture_and_dataflow.html` to interact with them.

1 What the design does

The accelerator computes **C = A × W**, where `W` is a stationary 4×4 matrix of 8-bit signed integers and `A` is a stream of 4-element activation vectors. Each vector produces one 4-element result vector of 32-bit signed integers, at a throughput of **one vector per clock cycle** once the pipeline is full.

It is a *weight-stationary* systolic array. That phrase describes the dataflow completely:

> **THE DATAFLOW IN ONE SENTENCE**
>
> **Weights enter from the north edge and stop; activations enter from the west and keep moving east; partial sums flow south and leave the bottom edge.**

The weight matrix is loaded once into the 16 processing elements — one 8-bit weight each — and then stays put. Activations stream through horizontally, and each processing element multiplies the activation passing through it by its resident weight and adds the result to the partial sum descending from above. By the time a partial sum leaves the bottom of a column it has accumulated all four products of a dot product.

Nothing in the array is ever re-fetched from memory during compute. That is the entire point of the architecture: a weight is read from SRAM once and then used for every activation vector in the job. Memory bandwidth — not arithmetic — is what limits real machine-learning accelerators, so minimising re-fetch is what makes the design fast.

Key numbers

Property	Value	Set by
Array size	4 rows × 4 columns = 16 processing elements	`ROWS`, `COLS`
Operand format	int8 (signed, −128…127)	`DATA_W = 8`
Accumulator format	int32 (signed)	`PSUM_W = 32`
Peak throughput	16 multiply-accumulates per clock	array size
Compute throughput	1 activation vector (4 MACs × 4 columns) per clock	no back-pressure
Result latency	7 cycles from activation address to result on the bus	`RESULT_LATENCY`
Store latency	8 cycles from activation address to write strobe	`STORE_LATENCY`
Job length	`num_vectors + 19` cycles	controller FSM
Activation memory	256 words × 32 bits (256 vectors)	`A_ADDR_W = 8`
Weight memory	4 words × 32 bits (one word per weight row)	`W_ADDR_W = 2`
Output memory	256 words × 128 bits	`O_ADDR_W = 8`

2 Repository and module map

```
verification_individual/
├── design_rtl/          13 synthesizable SystemVerilog files
├── test_benches/        13 testbenches + run_all_testbenches.ps1
├── indi_ver_rep/        verification report + raw QuestaSim transcripts
├── DOCS/                description index + detailed docs (this file lives here)
```

```
├── context_mkdwn/       derivations and design decisions
├── docs_images/         architecture_overview.jpeg
└── index.md             file locator
```

The 13 RTL modules

File	Module	Lines	Role
`MAC.sv`	`processing_element`	57	One weight-stationary multiply-accumulate cell
`array.sv`	`systolic_array`	63	4×4 grid of PEs — pure interconnect, no arithmetic
`delay_pipe.sv`	`delay_pipe`	31	Parameterised *N*-deep shift register; `DELAY=0` is a wire
`input_buf.sv`	`input_skew_buffer`	35	Dense vector → diagonal wavefront (row *r* delayed by *r*)
`output_skewbuf.sv`	`output_deskew_buffer`	24	Removes the column staircase (column *c* delayed by `COLS-1-c`)
`a_mem.sv`	`a_mem`	19	Activation SRAM, 256 × 32
`weight_mem.sv`	`weight_mem`	18	Weight SRAM, 4 × 32
`outmem.sv`	`output_mem`	18	Result SRAM, 256 × 128
`controller.sv`	`controller`	128	Phase FSM — owns durations, produces no addresses
`agu.sv`	`agu`	114	Address generator — owns addresses, holds no policy
`wrapper_array_buf.sv`	`array_buf_top`	100	**Level 1** — skew + array + de-skew
`wrapper_mem_arr_buf.sv`	`accelerator_top`	175	**Level 2** — level 1 + the three SRAMs
`wrapper_full_system.sv`	`accelerator_system_top`	205	**Level 3** — level 2 + FSM + AGU + arbitration

> **FILENAME ≠ MODULE NAME**
>
> Eight files carry a module whose name differs from the filename. Search by module name and use this table: `processing_element` → `MAC.sv`, `systolic_array` → `array.sv`, `input_skew_buffer` → `input_buf.sv`, `output_deskew_buffer` → `output_skewbuf.sv`, `output_mem` → `outmem.sv`, `array_buf_top` → `wrapper_array_buf.sv`, `accelerator_top` → `wrapper_mem_arr_buf.sv`, `accelerator_system_top` → `wrapper_full_system.sv`.

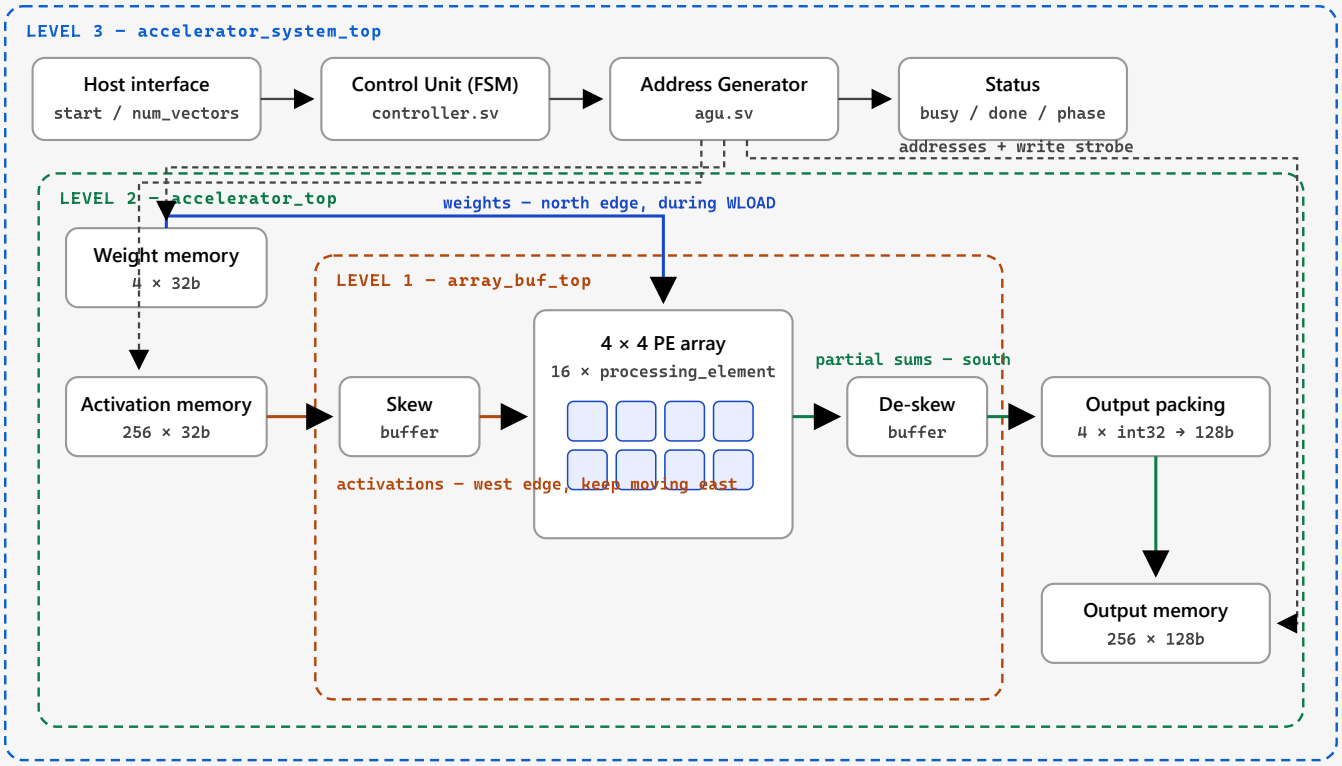
Instance hierarchy

```
accelerator_system_top                  wrapper_full_system.sv     LEVEL 3
├── controller                          controller.sv
├── agu                                 agu.sv
└── accelerator_top                     wrapper_mem_arr_buf.sv     LEVEL 2
    ├── a_mem                           a_mem.sv
    ├── weight_mem                      weight_mem.sv
    ├── output_mem                      outmem.sv
    └── array_buf_top                   wrapper_array_buf.sv       LEVEL 1
        ├── input_skew_buffer           input_buf.sv
        │   └── delay_pipe  × 2·ROWS    delay_pipe.sv
        ├── systolic_array              array.sv
        │   └── processing_element × 16 MAC.sv
        └── output_deskew_buffer        output_skewbuf.sv
            └── delay_pipe  × COLS      delay_pipe.sv
```

Twelve `delay_pipe` instances exist in total: eight inside the input skew buffer (four for data, four for the matching valid bits) and four inside the de-skew buffer.

3 Architecture: the three levels

Rather than one monolithic top module, the design closes at three abstraction levels, each in its own file. **Each level instantiates the level below rather than re-wiring the fabric.**



The three abstraction levels. Level 1 is the compute fabric with no memory and no sequencing. Level 2 adds the three SRAMs, the control re-timing and the output packing. Level 3 adds the FSM, the address generator and the port arbitration that decides whether the host or the AGU drives the shared SRAM address ports.

Why three levels instead of one

They cannot drift apart

The datapath verified by `tb_accelerator_top` is byte-for-byte the datapath running inside `accelerator_system_top`, because level 3 *instantiates* level 2 rather than re-declaring the same wires.

Failures localise instantly

Levels 1 and 2 have their own passing testbenches. A failure that appears only at level 3 is necessarily in the control plane — the FSM or the AGU — because everything below it is already proven.

Each level is independently usable

A user who wants to drive the fabric from their own sequencer instantiates level 1. A user who wants the memories but their own control instantiates level 2. Neither is forced to take the FSM.

No file overwrites another

Adding a fourth level means adding a fourth file that instantiates level 3. It never means editing `wrapper_full_system.sv`. This is a hard rule in this repository.

4 Word formats and data layout

Three SRAMs, three packing conventions. Getting a lane index wrong here transposes a matrix silently, so the mapping is worth stating precisely.

```
a_mem word       [31:24][23:16][15:8][7:0]
                  row 3  row 2  row 1 row 0      four int8 activations
                  → lane r feeds array ROW r

weight_mem[r]    [31:24][23:16][15:8][7:0]
                  col 3  col 2  col 1 col 0      four int8 weights of weight row r
                  → lane c feeds array COLUMN c
                  → address r holds weight row r

output_mem word  [127:96][95:64][63:32][31:0]
                  col 3   col 2  col 1  col 0    four int32 results
                  → lane c is the result of array COLUMN c
```

> **THE MENTAL MODEL**
>
> An `a_mem` word is one **column vector** of `A` laid flat — one number per array row. A `weight_mem` word is one **row** of `W` — one number per array column. An `output_mem` word is one **row** of `C`. The arithmetic the array performs is $C[i][c] = \Sigma_r A[i][r] \cdot W[r][c]$.

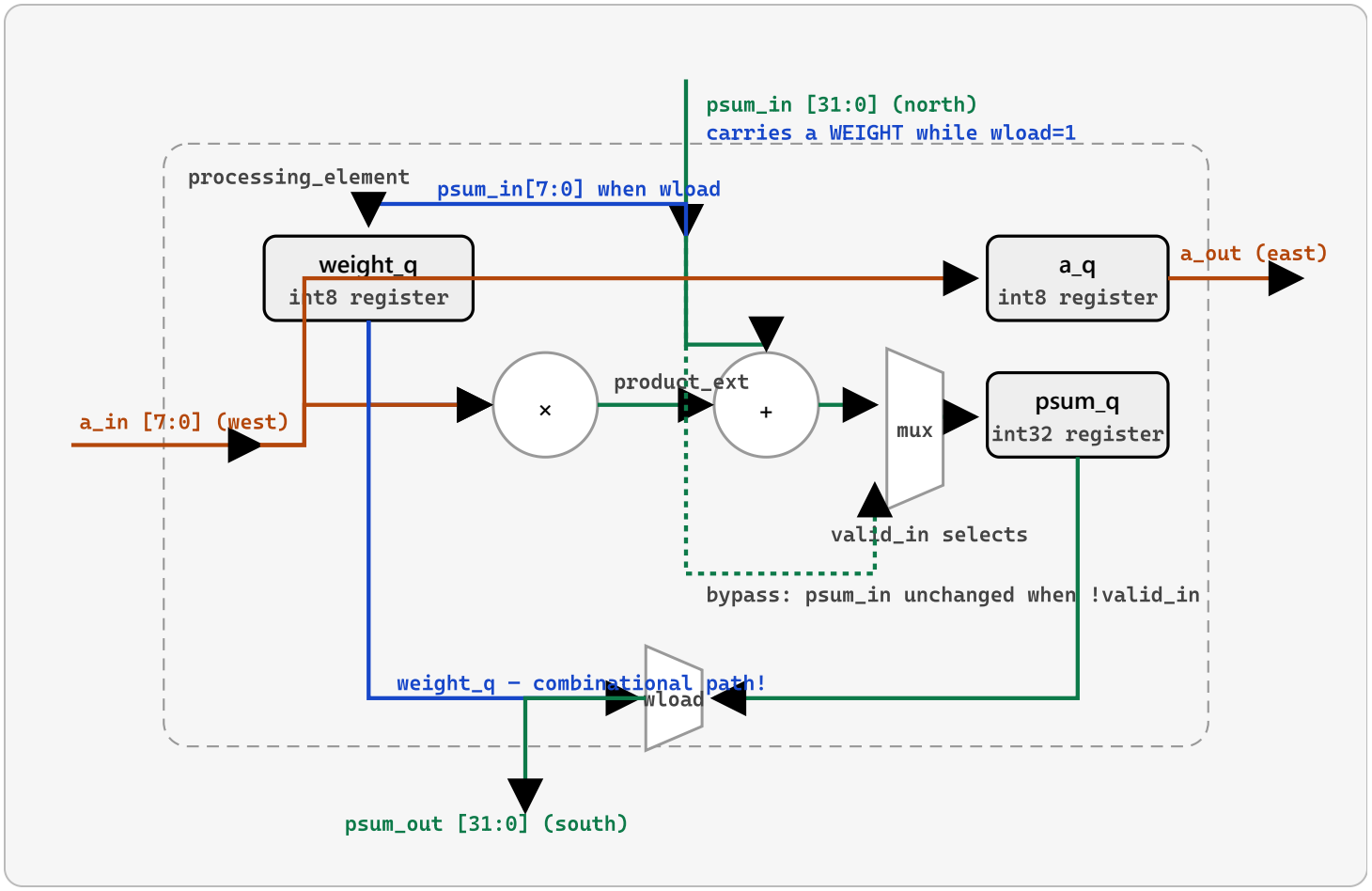
All three memories are the same single-port synchronous RAM primitive with three properties that propagate through the whole design:

1. **One address port, shared by read and write.** A write cycle is not a read cycle. This is what forces the port arbitration at level 3.
2. **One clock of read latency.** `rdata` is registered. This is the entire reason `accelerator_top` re-times `wload` and `valid_in`.
3. **Read-before-write on an address collision.** Both statements are non-blocking, so a read concurrent with a write to the same address returns the *old* contents.

None of the memories has a reset. Reads of unwritten addresses return `X` in simulation.

5 The processing element

`MAC.sv` is where every arithmetic operation in the design happens. Sixteen instances form the array; everything else is wiring, buffering and sequencing.



Inside one processing element. Four registers (`weight_q`, `a_q`, `valid_q`, `psum_q`), one multiplier, one adder, and two multiplexers. The dashed line is the bypass path taken when `valid_in = 0`. The blue path from `weight_q` to `psum_out` is **combinational** — that single wire is what makes a whole column behave as a downward shift register during weight load.

The source

```
// combinational
product_ext = 32'(weight_q * a_in);            // signed × signed, sign-extended
adder_out   = psum_in + product_ext;

// on every posedge (synchronous reset clears all four registers)
if (wload) weight_q <= psum_in[7:0];           // capture weight from the shared bus
a_q        <= a_in;
valid_q    <= valid_in;
psum_q     <= valid_in ? adder_out : psum_in;  // accumulate, or pass through

// outputs
a_out     = a_q;
valid_out = valid_q;
psum_out  = wload ? 32'(weight_q) : psum_q;    // combinational mux
```

Three things worth understanding

The vertical bus is time-multiplexed

During `wload` the north–south bus carries weights; during compute it carries partial sums. That is why `psum_in` is 32 bits wide but only its low 8 bits are ever captured as a weight. Sharing one bus for both saves a second 8-bit vertical network across the whole array — a real area saving that costs one mux per PE.

During wload, psum_out is combinational off weight_q

This is the single most consequential line in the design. Because `psum_out` bypasses the `psum_q` register during weight load, each PE presents its *currently held* weight to the PE below. At the next clock edge, PE row *r* captures what PE row *r*−1 held *before* the edge — a non-blocking assignment, so the pre-edge value. A column is therefore a plain downward shift register, and **the value fed first travels furthest**.

> **RULE 1 — WEIGHT ROWS LOAD BOTTOM-FIRST**
>
> To place weight row *r* into array row *r*, feed `W[ROWS-1]` first and `W[0]` last. The AGU implements this by counting the weight address **down**: 3, 2, 1, 0.
>
> Getting this wrong is silent. The design runs, `done` pulses, and plausible-looking 32-bit numbers appear in memory — computed against a vertically flipped weight matrix. There is no error flag, no timeout, no `X` propagation. Only a golden-model comparison against a *non-symmetric* weight matrix catches it, which is why every array-level testbench uses `$urandom_range` rather than a convenient hand-written matrix.

Valid gating is a bypass, not a stall

When `valid_in = 0` the PE registers `psum_in` unchanged instead of accumulating. Partial sums keep marching south at exactly one row per clock regardless of what `valid_in` is doing. A bubble in the activation stream therefore never *mixes* with real data — it produces a row of junk that flushes out on its own. There is no back-pressure anywhere in this design and no handshake to get wrong.

Verified by `tb_processing_element` — 1 566 checks, PASS. Directed vectors use hand-computed constants that owe nothing to the reference model: `100 + 3×5 = 115`, `(−128)×(−128) = +16384`, `50 + 127×(−2) = −204`, and weight `−3` during `wload` must appear on `psum_out` as `0xFFFFFFFD`.

6 delay_pipe — animated

`delay_pipe` is 31 lines and is the most-instantiated module in the design: twelve copies. It is a shift register that is `WIDTH` bits wide and `DELAY` stages deep, and its entire job is to make a value

arrive `DELAY` cycles later than it was presented.

```
generate
    if (DELAY == 0) begin : gen_no_delay
        assign dout = din;                        // a plain wire — no register at all
    end
    else begin : gen_with_delay
        logic [WIDTH-1:0] pipe [0:DELAY-1];
        always_ff @(posedge clk) begin
            if (!rst_n) for (i = 0; i < DELAY; i = i + 1) pipe[i] <= '0;
            else begin
                pipe[0] <= din;                       // newest value enters stage 0
                for (i = 1; i < DELAY; i = i + 1)
                    pipe[i] <= pipe[i-1];             // everything else shifts along
            end
        end
        assign dout = pipe[DELAY-1];               // oldest value leaves
    end
endgenerate
```

> **DELAY = 0 IS NOT A DEGENERATE CASE TO BE TOLERATED**
>
> It is used **deliberately** — by row 0 of the input skew buffer and by column `COLS-1` of the de-skew buffer. In those positions it must synthesise to a genuine wire with no register and no reset behaviour, which is exactly what the `gen_no_delay` branch produces. Writing this as `pipe[0:DELAY-1]` without the guard would declare a zero-element array and fail to elaborate.

The governing relation

Sampled just *before* the clock edge that will consume it, the pipe obeys one uniform relation for every depth including zero:

```
dout(cycle m) == din(cycle m - DELAY)
```

Sampled *after* an edge instead, a combinational wire and a one-deep register are indistinguishable — both show the value that was driven before that edge. That is why `tb_delay_pipe` samples on the **negative** edge, and why every testbench in this repository states its sampling convention in its header comment. Two of the three defects ever found in this work were violations of exactly this rule.

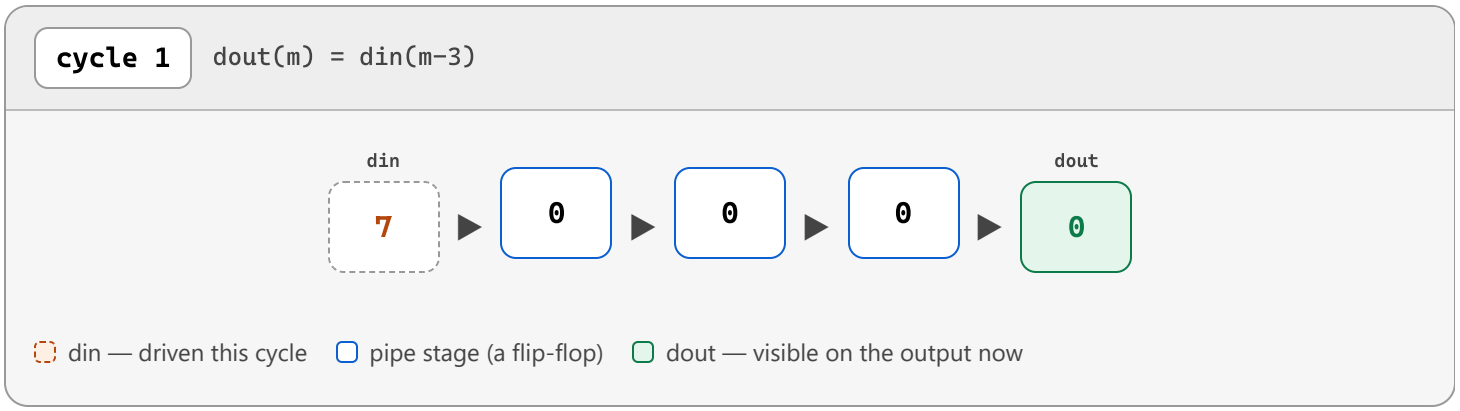


Figure 6.1 — a `delay_pipe` shifting. Change `DELAY` to see the depth change. At `DELAY = 0` the input box and the output box always hold the same value, because there is no register between them.

Complete trace — the value driven on `din` each cycle and what appears on `dout`, for all four depths:

cycle	din	dout D=0	dout D=1	dout D=2	dout D=3	relation check
1	7	7	0	0	0	dout(D=3) = din(−2) = 0 ✓
2	−3	−3	7	0	0	dout(D=3) = din(−1) = 0 ✓
3	12	12	−3	7	0	dout(D=3) = din(0) = 0 ✓
4	0	0	12	−3	7	dout(D=3) = din(1) = 7 ✓
5	25	25	0	12	−3	dout(D=3) = din(2) = −3 ✓
6	−8	−8	25	0	12	dout(D=3) = din(3) = 12 ✓
7	4	4	−8	25	0	dout(D=3) = din(4) = 0 ✓
8	19	19	4	−8	25	dout(D=3) = din(5) = 25 ✓
9	−11	−11	19	4	−8	dout(D=3) = din(6) = −8 ✓
10	6	6	−11	19	4	dout(D=3) = din(7) = 4 ✓
11	30	30	6	−11	19	dout(D=3) = din(8) = 19 ✓
12	2	2	30	6	−11	dout(D=3) = din(9) = −11 ✓

7 The input skew buffer — animated

The array needs its activations to arrive as a **diagonal wavefront**, not as a dense column. `input_skew_buffer` is what converts one into the other: it is four pairs of `delay_pipe`, row *r* with `DELAY = r`.

```
for (r = 0; r < ROWS; r = r + 1) begin : gen_input_skew
    delay_pipe #(.WIDTH(WIDTH), .DELAY(r)) u_a_delay ( ... dense_a_in[r]     -> skew_a_out[r]     );
    delay_pipe #(.WIDTH(1),     .DELAY(r)) u_v_delay ( ... dense_valid_in[r] -> skew_valid_out[r] );
end
```

The valid bit gets its own pipe with the *same* delay, so data and valid can never separate. That matters because the PE takes its accumulate-or-bypass decision from `valid_in` in the same cycle

it consumes `a_in`.

Why the delay has to be exactly *r*

Activations travel east one column per cycle, so the activation arriving at PE[*r*][*c*] at cycle *t* is the one that entered row *r* at cycle `t − c`. Unrolling the partial-sum recursion down a column gives

```
S_ROWS(t₀ + ROWS) = Σ_r  w[r][c] · a_in_left[r](t₀ + r − c)
```

For that sum to be a dot product, every row must contribute an element of the **same** activation vector — so `a_in_left[r]` must be delayed by *r* relative to row 0. Substituting `a_in_left[r](t) = dense_a[r](t − r)`:

```
a_in_left[r](t₀ + r − c) = dense_a[r](t₀ − c)
```

The `r` cancels. Every row now contributes the dense vector presented at cycle `t₀ − c`, and the sum is a genuine dot product. The skew delay does not *help* the array — it is what makes the array compute the right thing at all.

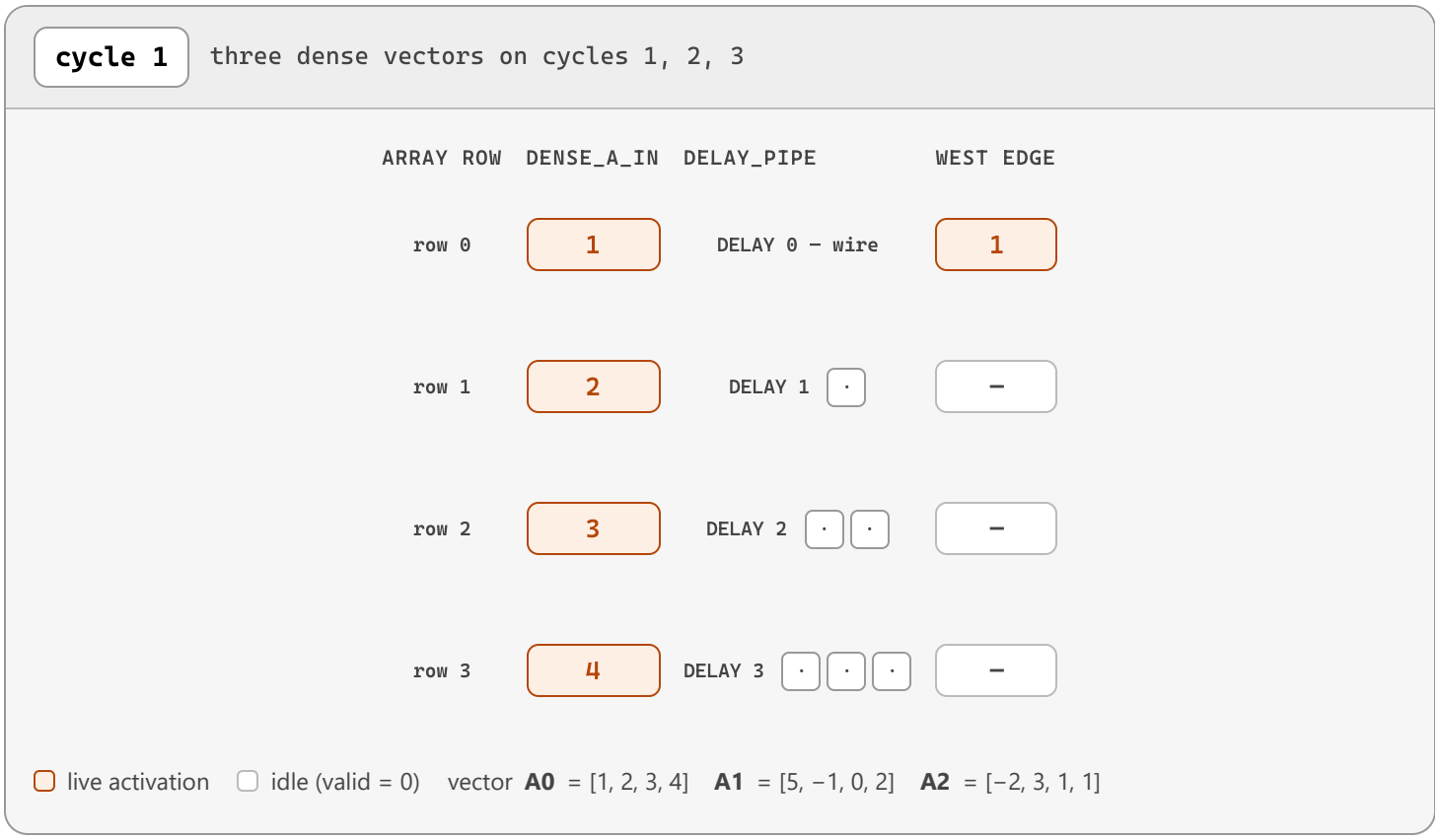


Figure 7.1 — dense in, diagonal out. On the left, three complete vectors are presented on three consecutive cycles. On the right, the array's west edge sees them staggered: row 0 immediately, row 1 one cycle later, row 3 three cycles later. Watch the `A0` elements form a diagonal line down the right-hand column.

Complete trace — what the array's west edge sees on every cycle (`—` means the valid bit is low):

cycle	dense_a_in (row 0..3)	west edge row 0	row 1	row 2	row 3	vectors in flight
1	A0 = [1, 2, 3, 4]	1 ·A0	—	—	—	A0
2	A1 = [5, −1, 0, 2]	5 ·A1	2 ·A0	—	—	A1 + A0
3	A2 = [−2, 3, 1, 1]	−2 ·A2	−1 ·A1	3 ·A0	—	A2 + A1 + A0
4	— (valid = 0)	—	3 ·A2	0 ·A1	4 ·A0	A2 + A1 + A0
5	— (valid = 0)	—	—	1 ·A2	2 ·A1	A2 + A1
6	— (valid = 0)	—	—	—	1 ·A2	A2
7	— (valid = 0)	—	—	—	—	—
8	— (valid = 0)	—	—	—	—	—
9	— (valid = 0)	—	—	—	—	—

8 The systolic array

`array.sv` contains no arithmetic whatsoever. It declares three interconnect meshes, instantiates 16 processing elements, and binds the boundaries. Every multiply and every add happens inside a PE.

```
logic signed [7:0]  a_net     [0:ROWS-1][0:COLS];    // horizontal, west → east
logic               valid_net [0:ROWS-1][0:COLS];    // horizontal, alongside a_net
logic signed [31:0] psum_net  [0:ROWS]  [0:COLS-1];  // vertical, north → south
```

Each mesh is sized **one larger than the grid** in its direction of travel. That is a small but important trick: it means the array boundaries are ordinary elements of the same array, so binding an input is a one-line `assign` rather than a special case.

```
for (r = 0; r < ROWS; r++) begin : left_bind
    assign a_net[r][0]     = a_in_left[r];        // west boundary
    assign valid_net[r][0] = valid_in_left[r];
end
for (c = 0; c < COLS; c++) begin : top_bind
    assign psum_net[0][c]  = psum_in_top[c];      // north boundary
end
for (r = 0; r < ROWS; r++) begin : row_gen
    for (c = 0; c < COLS; c++) begin : col_gen
        processing_element pe_inst (
            .a_in (a_net[r][c]),   .a_out    (a_net[r][c+1]),      // horizontal link
            .psum_in(psum_net[r][c]), .psum_out(psum_net[r+1][c]), // vertical link
            ... );
    end
end
for (c = 0; c < COLS; c++) begin : bottom_bind
    assign psum_out_bottom[c] = psum_net[ROWS][c];  // south boundary
end
```

The instance name of PE row 2, column 3 in a waveform viewer is therefore `u_array.row_gen[2].col_gen[3].pe_inst`. Naming the generate blocks is not cosmetic — an unnamed generate block gets a tool-assigned name like `genblk1`, which makes hierarchical references fragile across tool versions.

Column latency

With activations skewed so that row *r* is delayed by *r*, a dense vector consumed at cycle *n* produces on column *c* a result observable immediately after cycle

```
n + (ROWS-1) + c
```

`ROWS-1` is one register stage per PE row; the extra `+c` is the horizontal travel time of the activations. Sanity check for `c = 0`, `ROWS = 4`: four PEs, four register stages, first result visible after edge `n+3`. ✓

That `+c` is the staircase — column 0 finishes first, column 3 finishes three cycles later — and removing it is the de-skew buffer's only job.

9 Matrix multiply, cycle by cycle — animated

This is the centrepiece figure. It runs a complete job on the level-1 fabric — `input_skew_buffer` → `systolic_array` → `output_deskew_buffer` — through all four phases, and it is driven by a **cycle-accurate JavaScript model of the RTL**: the same four registers per PE, the same combinational `psum_out = wload ? weight_q : psum_q` mux, the same non-blocking update semantics.

The problem being solved

Weight matrix W (stationary)

```
       c0   c1   c2   c3
r0 [   2   -1    3    0 ]
r1 [   1    4   -2    5 ]
r2 [  -3    2    1    1 ]
r3 [   0    1    2   -4 ]
```

Activation vectors A (streamed)

```
       r0   r1   r2   r3
A0 [   1    2    3    4 ]
A1 [   5   -1    0    2 ]
A2 [  -2    3    1    1 ]
```

Expected result, computed independently as `C[i][c] = Σ_r A[i][r] · W[r][c]` — this is exactly the golden model the testbenches use, and the animation checks itself against it:

```
C0 = [  -5   17   10   -3 ]
C1 = [   9   -7   21  -13 ]
C2 = [  -4   17   -9   12 ]
```

The four phases

Phase	Cycles	What is happening
`WLOAD`	1–4	`wload = 1`. Weight rows are driven onto the north edge **bottom-first**: W[3], W[2], W[1], W[0]. The vertical bus is a downward shift register, so the first row fed travels furthest.
`FLUSH`	5–10	`wload = 0`, zeros on the north edge, `valid = 0`. The `psum_q` registers latched weight values during WLOAD; this pushes that residue out of the bottom.
`COMPUTE`	11–13	One dense activation vector per clock with `valid = 1`. A0 at cycle 11, A1 at 12, A2 at 13.
`DRAIN`	14–21	The last vector is still travelling through the array, the de-skew buffer and the packing

Phase	Cycles	What is happening
		logic. C0 lands after cycle 17, C1 after 18, C2 after 19.

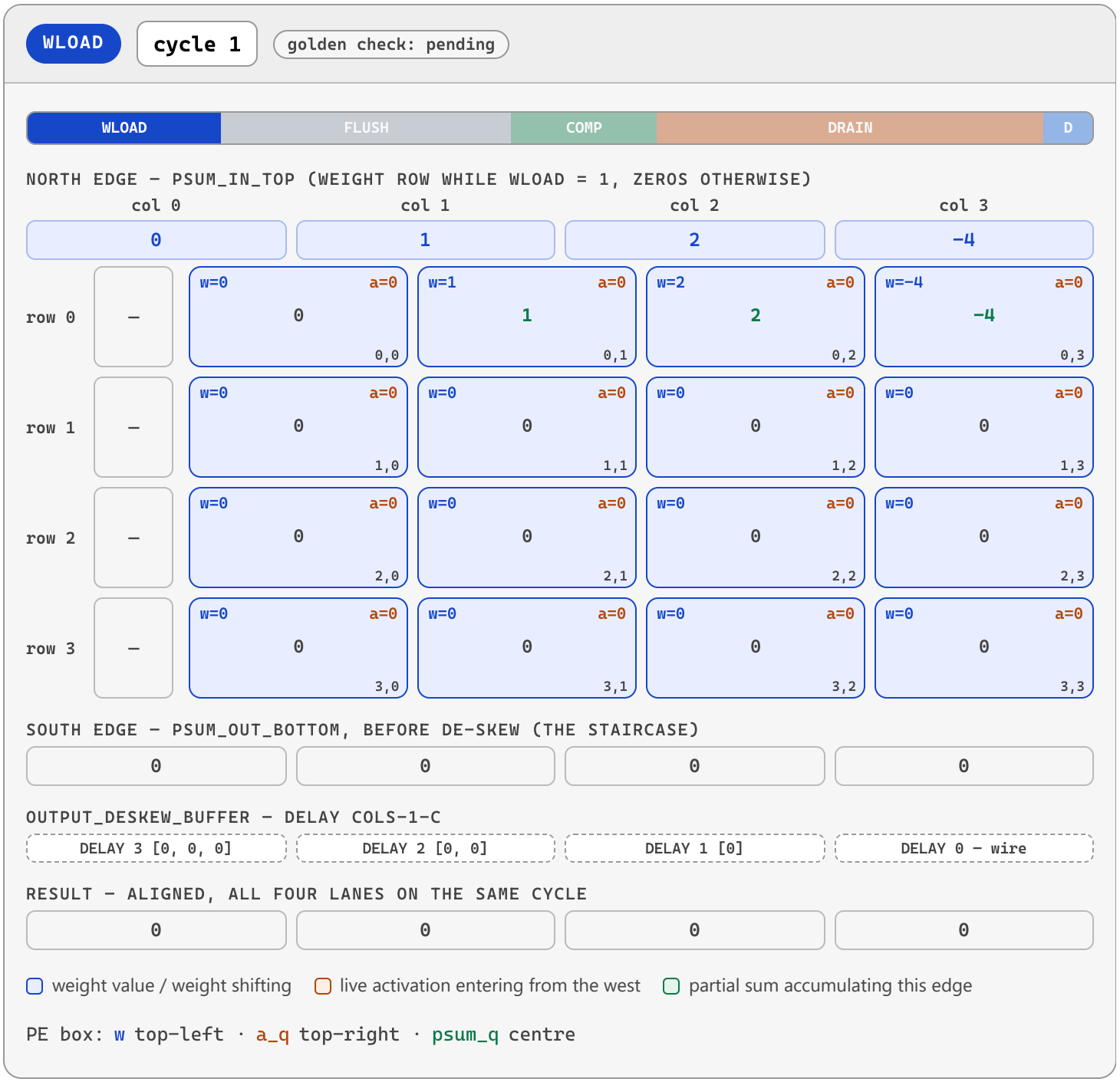


Figure 9.1 — a complete 3-vector job. Step through cycle by cycle. Watch four distinct things: (1) during WLOAD the weights march *down* the columns, so W[3] fed first ends up in the bottom row; (2) during COMPUTE the activations enter staggered from the west; (3) the partial sums accumulate downward, one row per cycle, each column finishing one cycle later than the one to its left; (4) the de-skew pipes remove that stagger so all four results of a vector appear on `result` on the same cycle.

Complete cycle-by-cycle trace of the whole job. `W-row` is the weight row on the north edge, `west` is the array's west edge after skewing, `bottom` is `psum_out_bottom` before de-skew, `result` is the aligned output. Green rows carry a finished result vector.

cycle	phase	north edge (psum_in_top)	west edge (after skew)	bottom (psum_out_bottom)	result (after de-skew)	note
1	WLOAD	[0, 1, 2, -4]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	weight row W[3] → shifts down; lands in array row 3
2	WLOAD	[-3, 2, 1, 1]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	weight row W[2] → shifts down; lands in array row 2
3	WLOAD	[1, 4, -2, 5]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	weight row W[1] → shifts down; lands in array row 1
4	WLOAD	[2, -1, 3, 0]	[–, –, –, –]	[0, 1, 2, -4]	[0, 0, 0, -4]	weight row W[0] → shifts down; lands in array row 0
5	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[-3, 2, 1, 1]	[0, 0, 2, 1]	zeros pushed through, valid = 0 – clearing weight residue
6	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[1, 4, -2, 5]	[0, 1, 1, 5]	zeros pushed through, valid = 0 – clearing weight residue
7	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[2, -1, 3, 0]	[0, 2, -2, 0]	zeros pushed through, valid = 0 – clearing weight residue
8	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[-3, 4, 3, 0]	zeros pushed through, valid = 0 – clearing weight residue
9	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[1, -1, 0, 0]	zeros pushed through, valid = 0 – clearing weight residue
10	FLUSH	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[2, 0, 0, 0]	zeros pushed through, valid = 0 – clearing weight residue
11	COMPUTE	[0, 0, 0, 0]	[1, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	vector A0 consumed at this edge
12	COMPUTE	[0, 0, 0, 0]	[5, 2, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	vector A1 consumed at this edge
13	COMPUTE	[0, 0, 0, 0]	[-2, -1, 3, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	vector A2 consumed at this edge
14	DRAIN	[0, 0, 0, 0]	[–, 3, 0, 4]	[-5, 0, 0, 0]	[0, 0, 0, 0]	pipeline draining
15	DRAIN	[0, 0, 0, 0]	[–, –, 1, 2]	[9, 17, 0, 0]	[0, 0, 0, 0]	pipeline draining
16	DRAIN	[0, 0, 0, 0]	[–, –, –, 1]	[-4, -7, 10, 0]	[0, 0, 0, 0]	pipeline draining
17	DRAIN	[0, 0, 0, 0]	[–, –, –, –]	[0, 17, 21, -3]	**[-5, 17, 10, -3]**	C0 complete – golden [-5, 17, 10, -3]
18	DRAIN	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, -9, -13]	**[9, -7, 21, -13]**	C1 complete – golden [9, -7, 21, -13]
19	DRAIN	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 12]	**[-4, 17, -9, 12]**	C2 complete – golden [-4, 17, -9, 12]
20	DRAIN	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	pipeline draining
21	DRAIN	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	pipeline draining
22	DONE	[0, 0, 0, 0]	[–, –, –, –]	[0, 0, 0, 0]	[0, 0, 0, 0]	

Reading the trace

- **Cycles 1–4, the `W-row` column.** The order is 3, 2, 1, 0. After cycle 4 every PE holds its correct weight — check the weight values in the animation against the W matrix above.
- **Cycles 1–4, the `bottom` column.** Weight values appear here. That is the residue the FLUSH phase exists to clear, and it is why the result bus must not be trusted until compute begins.
- **Cycle 11.** A0 enters. Only `west[0]` is live — rows 1, 2 and 3 are still inside their skew pipes.
- **Cycle 14.** `bottom[0]` becomes −5, which is `C0[0]`. That is `11 + (ROWS−1) + 0 = 14`. ✓
- **Cycles 15, 16, 17.** `bottom[1]`, `bottom[2]`, `bottom[3]` follow one cycle apart — the staircase.
- **Cycle 17.** All four elements of C0 appear simultaneously on `result`. That is `11 + ROWS + COLS − 2 = 17`. ✓ The de-skew delay `COLS−1−c` exactly cancels the array's `+c`.

> **WHY FLUSH IS BELT-AND-BRACES, AND WHY IT STAYS ANYWAY**
>
> Strictly, FLUSH is **not required for correctness**. Partial sums march south at exactly one row per cycle, so weight residue and a real accumulation always occupy different pipeline slots — they can never mix. FLUSH exists so the result bus is *visibly* quiet before compute starts, which is what makes the "quiet before compute" assertions and the waveforms meaningful.
>
> This is recorded explicitly so that a future reader neither deletes it as pointless nor treats it as untouchable.

A full 4×4 × 4×4 matrix multiply

The animation above streams three activation vectors. Nothing about the hardware changes if we stream **four** and read the activation stream itself as a 4×4 matrix: the accelerator then computes an ordinary **4×4 × 4×4 = 4×4** matrix product, one result *row* per activation row. This is exactly the case the re-verification testbench `tb_accelerator_system_top` drives as a directed check (section **T5**), with hand-verifiable numbers:

```
   A (4×4)            W (4×4)              C = A × W (4×4)
[ 1 0 0 0 ]       [  1  2  3  4 ]       [  1  2  3  4 ]   ← A row0 selects W row0
[ 0 1 0 0 ]   ×   [  5  6  7  8 ]   =   [  5  6  7  8 ]   ← A row1 selects W row1
[ 0 0 1 0 ]       [  9 10 11 12 ]       [  9 10 11 12 ]   ← A row2 selects W row2
[ 1 1 1 1 ]       [ 13 14 15 16 ]       [ 28 32 36 40 ]   ← A row3 = column sums
```

Each result row is one activation vector multiplied by the whole stationary weight matrix, `C[i][c] = Σ_r A[i][r]·W[r][c]`. Because **W** is loaded once and reused for every activation row, a 4×4 by 4×4 multiply costs one weight-load followed by four back-to-back compute cycles — the same weight-stationary economics that let the design multiply a single weight matrix against a stream of up to 256 activation vectors (the depth of `a_mem`). The identity-style rows above make the answer checkable by eye: rows 0–2 of **C** reproduce weight rows 0–2, and the all-ones activation row 3 yields the four column sums `[28 32 36 40]`.

A second, fully general 4×4 × 4×4 example

The identity-style case is easy to read but is a special one. Here is a completely general product of two arbitrary 4×4 integer matrices — the same computation the accelerator performs, one activation row streamed per clock, verified against the golden model. Every entry is `C[i][c] = Σ_r A[i][r]·W[r][c]`:

```
        A (4×4)                    W (4×4)                      C = A × W (4×4)
[  2  -1   3   0 ]        [  1   2  -1   3 ]              [  17    9    0   -4 ]
[  1   4  -2   5 ]   ×    [  0  -2   4   1 ]      =       [ -14    7   11   23 ]
[ -3   2   1   1 ]        [  5   1   2  -3 ]              [   1   -6   13   -8 ]
[  0   1   2  -4 ]        [ -1   3   0   2 ]              [  14  -12    8  -13 ]

worked entries
  C[0][0] = 2·1 + (−1)·0 + 3·5 + 0·(−1)          = 17
  C[1][2] = 1·(−1) + 4·4 + (−2)·2 + 5·0          = 11
  C[3][3] = 0·3 + 1·1 + 2·(−3) + (−4)·2          = −13
```

To run this on the hardware the host writes the four **W** rows into `weight_mem` (addresses 0–3, one row per word), preloads the four **A** rows into `a_mem`, then pulses `start` with `num_vectors = 4`. Sixteen cycles after the last activation address the whole 4×4 result matrix sits in `output_mem`, one 128-bit word per result row. Both this example and the identity one are checked as directed cases by the re-verification testbench `tb_accelerator_system_top` (sections T5 and T6) — see `verification_recheck/verification_recheck_report.md`.

10 The output de-skew buffer

Twenty-four lines, and it does one thing: undo the `+c` staircase that section 8 derived.

```
for (c = 0; c < COLS; c = c + 1) begin : gen_output_deskew
    delay_pipe #(.WIDTH(WIDTH), .DELAY(COLS-1-c)) u_psum_delay (
        .din (psum_in[c]), .dout (aligned_out[c]) );
end
```

Column 0 finishes first and is therefore delayed the most (`COLS-1 = 3`); column 3 finishes last and passes through a plain wire (`DELAY = 0`). The two effects cancel exactly:

```
column c result appears at   n + (ROWS-1) + c
de-skew delay                          + (COLS-1-c)
                             ──────────────────────
aligned at                   n + ROWS + COLS − 2      ← independent of c
```

Because the result no longer depends on *c*, all four lanes emerge on the same cycle and the packing logic in `accelerator_top` can simply concatenate them into one 128-bit word. Any other de-skew

depth would produce a word containing four results belonging to four *different* activation vectors — which, again, would be silent.

Verified by `tb_output_deskew_buffer` — 188 checks, including an explicit staircase-in / single-cycle-out alignment test and a negative check that the output is **empty on every cycle** until the last column has been fed.

11 The memories

`a_mem.sv`, `weight_mem.sv` and `outmem.sv` are the **same** primitive at three geometries. The entire body of each is four lines:

```
always_ff @(posedge clk) begin
    if (we) mem[addr] <= wdata;
    rdata <= mem[addr];
end
```

Module	File	Depth × Width	Address width	Word contents
`a_mem`	`a_mem.sv`	256 × 32	8	4 × int8, lane *r* = activation for array row *r*
`weight_mem`	`weight_mem.sv`	4 × 32	2	4 × int8, lane *c* = weight for array column *c*
`output_mem`	`outmem.sv`	256 × 128	8	4 × int32, lane *c* = result of array column *c*

Why weight_mem is 4 × 32 while a_mem is 256 × 32

The three memories share one primitive, but their *depths* differ by 64×, and that asymmetry is a direct consequence of the **weight-stationary** dataflow — not an arbitrary choice.

- **weight_mem holds one whole matrix, once.** The array has exactly `ROWS = 4` rows, and each 32-bit word packs the four int8 weights of one weight row (lane *c* = column *c*). Four rows ⇒ four words ⇒ `ADDR_W = 2`. The weights are loaded into the 16 PEs once at the start of a job and then **stay resident** — nothing in the array is ever re-fetched from weight_mem during compute. There is simply nothing more to store: a 4×4 int8 matrix *is* 4 × 32 bits, so a deeper weight memory would be dead silicon.
- **a_mem holds a whole stream of activation vectors.** Activations are the data that *moves*. One 32-bit a_mem word is one activation vector (lane *r* = row *r*), and a job streams `num_vectors` of them, one per clock. 256 words = up to 256 activation vectors multiplied against a **single** weight load, without reloading weights. That amortisation — pay the weight-load cost once, reuse it over as many activation rows as the activation memory can hold — is the entire economic point of a weight-stationary array.

Put arithmetically: weight traffic is `ROWS × COLS` bytes **per job**, while activation traffic is `ROWS` bytes **per vector**. Sizing the two memories to those rates keeps the expensive resource — memory bandwidth — busy on activations (re-read every cycle) rather than on weights (read once). `output_mem` mirrors `a_mem`'s depth (256 × 128) because it holds one result vector per activation vector.

Why this exact coding style matters

This is the canonical inference template for a **single-port synchronous RAM with registered read data**. Every FPGA and ASIC synthesis tool recognises it and maps it onto a hard block — a Block RAM on FPGA, a compiled SRAM macro on ASIC — instead of building 8 192 flip-flops out of logic. Three details are load-bearing:

- **The read is unconditional.** `rdata <= mem[addr];` sits outside the `if (we)`. Guarding it would infer a read-enable that many memory macros do not have.
- **There is no reset.** A reset on the data array would prevent hard-block inference entirely, because real SRAM macros cannot be cleared in one cycle. The cost is that reads of unwritten addresses return `X` in simulation — which is arguably a feature, since it makes reading uninitialised memory visible rather than silently returning zero.
- **Both assignments are non-blocking.** That is what produces **read-before-write** behaviour: a read concurrent with a write to the same address returns the *old* contents. Swapping to blocking assignments would produce write-first behaviour and change the design's semantics.

Verified by `tb_a_mem` (538 checks), `tb_weight_mem` (125) and `tb_output_mem` (455) — all PASS. All three properties are checked explicitly, plus byte-lane integrity tests that match the packing conventions above. `tb_output_mem` is where defect #1 was found: `128'(-32'sd1)` sign-extends to 128 bits of ones, but the actual part-select `rd[32+:32]` is unsigned and zero-extends. The check was correct; the expected value was wrong.

12 Control plane: FSM and AGU

The control plane is split along one clean line, taken directly from the architecture diagram:

controller.sv — durations only

Knows how long each phase lasts. Emits four phase enables and three status bits. Produces **no addresses at all**.

agu.sv — addresses only

Knows what address belongs to each phase. Holds **no policy** — it never decides when a phase starts or ends, it just counts while enabled.

Neither knows about SRAM read latency. That is `accelerator_top`'s job, and keeping it there is what lets both control modules be written and tested without reference to the memories.

12.1 The phase FSM

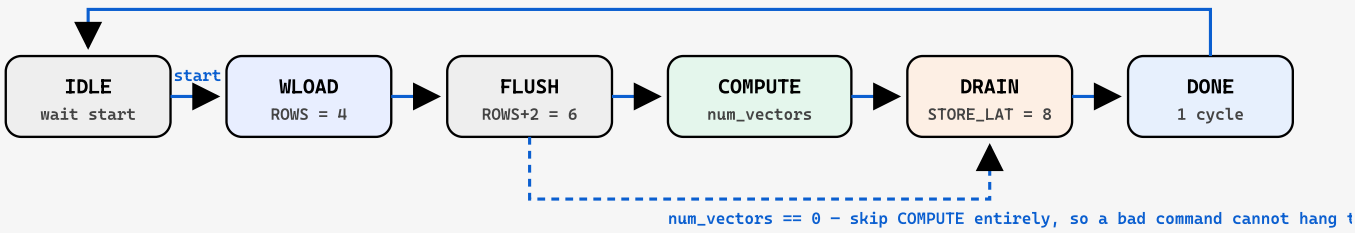


Figure 12.1 — the controller's six states. `busy` covers WLOAD…DRAIN. `done` is a single-cycle pulse fired in DONE, by which point `busy` is already low. `start` is sampled only in IDLE and is ignored while busy, so a stray pulse mid-job changes nothing.

The state register, counter and vector-count latch are the whole module:

```
PH_WLOAD:  if (cnt == VEC_CNT_W'(ROWS-1))          { cnt <= 0; state <= PH_FLUSH;   } else cnt++;
PH_FLUSH:  if (cnt == VEC_CNT_W'(FLUSH_CYCLES-1))  { cnt <= 0;
               state <= (vec_n == 0) ? PH_DRAIN : PH_COMPUTE;                    } else cnt++;
PH_COMPUTE:if (cnt == (vec_n - 1'b1))              { cnt <= 0; state <= PH_DRAIN;   } else cnt++;
PH_DRAIN:  if (cnt == VEC_CNT_W'(STORE_LATENCY-1)) { cnt <= 0; state <= PH_DONE;    } else cnt++;
```

> **WHY NUM_VECTORS = 0 IS HANDLED EXPLICITLY**
>
> `COMPUTE` exits on `cnt == vec_n - 1`. With `vec_n = 0` that comparison is against `8'hFF`, so the FSM would sit in COMPUTE for 256 cycles issuing garbage addresses and 256 write strobes — actively corrupting memory. The one-line guard in FLUSH turns a malformed command from destructive into harmless. It costs one comparator.

Verified by `tb_controller` — 85 checks. Every phase length is measured by counting clocks with that phase's enable high, for `num_vectors ∈ {0, 1, 3, 4, 6, 8, 16}`, plus the total job length measured independently. This testbench is where defect #3 was found: the testbench cleared its phase counters *after* the `start` pulse, discarding the first WLOAD cycle and measuring 3 instead of 4. The RTL was correct.

12.2 The address generator

Weight address — counts down

```
if (!wload_phase) w_cnt <= W_ADDR_W'(ROWS-1);   // re-armed whenever idle
else              w_cnt <= w_cnt - 1'b1;         // 3, 2, 1, 0
```

Re-arming while idle means the very first WLOAD cycle already presents `ROWS-1`, and a second job starts from `ROWS-1` again rather than continuing from wherever the last job left off.

Activation address — counts up

```
if (!compute_phase) act_cnt <= '0;
else                act_cnt <= act_cnt + 1'b1;
assign act_addr = act_base + act_cnt;
```

Output address and write strobe — a delay line, not a calculation

A result cannot be written when its activation address is issued; it is still travelling through the SRAM, the skew buffer, the PE column, the de-skew buffer and the packing logic. The obvious implementation is "*assert* `o_we` *when the counter reaches n + 8*". This design does something else:

```
out_we_pipe[0]   <= compute_phase;
out_addr_pipe[0] <= out_base + O_ADDR_W'(act_cnt);
for (i = 1; i < STORE_LATENCY; i++) begin
    out_we_pipe[i]   <= out_we_pipe[i-1];
    out_addr_pipe[i] <= out_addr_pipe[i-1];
end
assign out_we   = out_we_pipe[STORE_LATENCY-1];
assign out_addr = out_addr_pipe[STORE_LATENCY-1];
```

> **STRUCTURAL GUARANTEES BEAT COMPUTED ONES**
>
> Eight flip-flops buy something an equation cannot: the write strobe **structurally cannot** drift away from the data it belongs to. Changing the datapath depth means changing one parameter rather than re-deriving an equation and hoping every user of it was updated.
>
> The testbench still checks it — but it is checking that a shift register was wired up, not that an equation was got right, and those two things fail in very different ways.

Verified by `tb_agu` — 62 checks, driven by **testbench-generated** phase enables rather than a real controller. That is deliberate: if the FSM held a phase one cycle too long *and* the AGU miscounted in the compensating direction, a combined test would pass. The write-strobe scoreboard schedules each expected `{strobe, address}` at a specific future cycle and counts **extra**, **missing** and **misaddressed** strobes separately — a naive "did we see 6 writes?" check would pass a pipeline that drops its last entry and emits a spurious one somewhere else.

12.3 Port arbitration at level 3

Every SRAM is single-port, so exactly one master may drive it:

`busy`	Owner	Behaviour
0	Host	Preload activation vectors and weight rows; read results back. The host side of `output_mem` is read-only.
1	AGU	Host write strobes are **forced to zero**, so a stray host access cannot corrupt a running job.

This is verified adversarially. `tb_accelerator_system_top` run 2 drives `0xDEADBEEF` into the weight memory write port and `0xBAADF00D` into the activation memory write port on **every cycle of a live job**, and requires the results to be bit-identical to the clean run. That is what proves the arbitration actually masks writes, rather than merely appearing to when nobody is pushing.

13 Timing derivations

Notation

- **Cycle *n*** is the *n*-th positive clock edge.
- A signal **issued at cycle *n*** is the value present just *before* edge *n* — the value that edge samples.
- A signal **observed at cycle *n*** is the value present just *after* edge *n*.

The distinction between *issued* and *observed* is the whole ballgame; conflating them produced two of the three defects found in this work.

The chain, one step at a time

#	Step	Adds	Running total
1	Down a PE column: one register stage per row	`ROWS-1` = 3	3
2	East to column *c*: horizontal travel	`+c`	`3 + c`
3	De-skew pipe for column *c*	`COLS-1-c`	**6**, independent of *c*
4	SRAM read latency on `a_mem`	+1	**7** = `RESULT_LATENCY`
5	The write strobe must be high the cycle *after* the data is observable	+1	**8** = `STORE_LATENCY`

Summary of every latency constant

Constant	Value (4×4)	Formula	Declared in
Array column latency	3 + *c*	`(ROWS-1) + c`	`array.sv` (emergent)
De-skew delay	3 − *c*	`COLS-1-c`	`output_skewbuf.sv`
Level 1 result latency	6	`ROWS + COLS - 2`	`wrapper_array_buf.sv`
Level 2 result latency	7	`1 + ROWS + COLS - 2`	`wrapper_mem_arr_buf.sv`
Level 2 store latency	8	`RESULT_LATENCY + 1`	`wrapper_mem_arr_buf.sv`
AGU write-pipe depth	8	`STORE_LATENCY`	`agu.sv`
Controller `DRAIN`	8	`STORE_LATENCY`	`controller.sv`
Total job length	`N + 19`	`ROWS + FLUSH_CYCLES + N + STORE_LATENCY + 1`	emergent

The last three rows are the *same number*, and all three are derived from the single `STORE_LATENCY` parameter rather than written out three times. If the datapath depth ever changes, one parameter moves and the FSM, the AGU and the wrapper all follow.

Why control is re-timed inside the wrapper

Both `weight_mem` and `a_mem` have one clock of read latency, so the data belonging to an address issued at cycle *m* arrives at the array at cycle *m*+1. `wload` and `valid_in` must be high at *m*+1, not *m*. There were two ways to arrange that: make every caller pre-delay its control signals, or re-time inside the wrapper. `accelerator_top` re-times:

```
wload_q <= wload;
valid_q <= valid_in;
```

> **RULE 2 — CONTROL TRAVELS WITH ITS ADDRESS**
>
> Issue `wload` and `valid_in` in the **same** cycle as the address they belong to. Never pre-delay them at the caller. The AGU follows the same rule, which is why `agu.sv` contains no knowledge of SRAM latency at all.

> **RULE 3 — FIXED LATENCY, NO BACK-PRESSURE**
>
> There is no ready/valid handshake anywhere in this design. The host must respect the latency numbers above; the design will not stall to wait for it and will not signal congestion.

14 End-to-end data flow

Putting all three levels together, here is the complete path a single number takes.

A weight, from memory to its PE

1. Host writes `weight_mem[r]` while `!busy` — one 32-bit word holding the four int8 weights of weight row *r*, lane *c* for column *c*.
2. FSM enters WLOAD. AGU drives `w_addr = 3` and `wload = 1` in the same cycle.
3. One cycle later `w_rdata` returns weight row 3, and `wload_q` — the re-timed strobe — is high.
4. `drive_array_top` sign-extends each int8 lane to 32 bits and drives it onto the array's north edge: `psum_top[c] = PSUM_W'(signed'(w_rdata[c*8 +: 8]))`.
5. The vertical bus shifts down. Over four cycles W[3] travels to row 3, W[2] to row 2, and so on.

An activation, from memory to a result in memory

1. Host writes `a_mem[act_base + i]` — one 32-bit word, lane *r* for array row *r*.
2. Cycle *m*: AGU issues `act_addr` and `valid_in = 1` together, and pushes `{1, out_base+i}` into stage 0 of its 8-deep write pipe.
3. Cycle *m*+1: `a_rdata` returns; `unpack_activations` splits it into four signed int8 values; `valid_q` is high. The vector is consumed by the array at this edge.
4. Cycles *m*+1 … *m*+7: the skew buffer staggers the four elements; the array accumulates one row per cycle down each column; the de-skew buffer removes the column stagger.
5. Just after cycle *m*+7: all four int32 results are on `result[0:3]`. `pack_output` concatenates them into one 128-bit `o_wdata`.
6. Cycle *m*+8: `{1, out_base+i}` reaches the end of the AGU's write pipe. `o_we` is high, and `output_mem` samples `o_wdata` at that edge.
7. After `done`: the host reads it back through `host_o_addr` / `o_rdata`, and gets one 128-bit word containing C[*i*][0..3].

Driving the design

```
1.  hold rst_n low, then release
2.  while (!busy)  write activation vectors and weight rows via the host_* ports
3.  pulse start with num_vectors / act_base / out_base valid
4.  wait for the done pulse            // single cycle; busy is already low
5.  while (!busy)  read results via host_o_addr / o_rdata
```

Job length is `num_vectors + 19` cycles. `start` is ignored while busy. `num_vectors = 0` is legal and does not hang.

15 Verification summary

Testbench	DUT	Checks	Result
`tb_delay_pipe`	`delay_pipe`	127	`PASS`
`tb_processing_element`	`processing_element`	1 566	`PASS`
`tb_a_mem`	`a_mem`	538	`PASS`
`tb_weight_mem`	`weight_mem`	125	`PASS`
`tb_output_mem`	`output_mem`	455	`PASS`
`tb_input_skew_buffer`	`input_skew_buffer`	366	`PASS`
`tb_output_deskew_buffer`	`output_deskew_buffer`	188	`PASS`
`tb_systolic_array`	`systolic_array`	37	`PASS`
`tb_array_buf_top`	`array_buf_top` (L1)	76	`PASS`
`tb_controller`	`controller`	85	`PASS`
`tb_agu`	`agu`	62	`PASS`
`tb_accelerator_top`	`accelerator_top` (L2)	76	`PASS`
`tb_accelerator_system_top`	`accelerator_system_top` (L3)	201	`PASS`
Total	**13 DUTs**	**3 902**	`13/13`

Method, in four points

1. **Golden models are algorithmic, not RTL mirrors.** From `tb_systolic_array` upwards, expected values are `C = A × W` computed by nested loops. Nothing about the expected *value* comes from the RTL — it is the specification written as arithmetic. Only the expected *cycle* is derived from pipeline structure, and that is a stated limitation.
2. **Every testbench declares its sampling convention in its header.** Convention A samples after the edge (registered outputs); Convention B samples on the negative edge, before the edge that consumes the value (delay lines and address buses). Mixing them is what caused two of the three defects.
3. **Negative checks are mandatory.** A passing test proves the design does something; a negative check proves the testbench would notice if it did not. Examples: the result must **not** be present one cycle early; the de-skew output must be **empty** until the last column is fed; a word past the end of a short run must be **untouched**; `start` during a job must change **nothing**.
4. **A passing test must still be readable.** The first version printed a banner and nothing else — `tb_processing_element` ran 1 566 checks and emitted three lines. All 13 now carry an identical reporting block and print decoded stimulus, a per-transaction trace with expected values alongside, and a per-section summary. The transcripts went from ~550 lines to ~2 900.

The three defects

#	Where	Cause
1	`tb_output_mem`	`128'(-32'sd1)` sign-extends; the actual part-select `rd[32+:32]` is unsigned and zero-extends
2	`tb_agu`	Sampled the address bus *after* the edge that consumes it
3	`tb_controller`	Cleared phase counters after the `start` pulse, discarding the first WLOAD cycle

All three were in testbench code. None was in the RTL. That is a genuine result for a design of this size — but it is a statement about these tests, not a proof of correctness.

16 Known limitations

Stated plainly, because a green regression that overstates its own scope is worse than a red one.

- **Verified at `ROWS = COLS = 4` only.** `systolic_array`'s ports are hard-coded `[7:0]` / `[31:0]`, so the wrappers' `DATA_W` and `PSUM_W` parameters are effectively fixed at 8 and 32.
- **No overflow protection on the 32-bit accumulator.** Unreachable at 4×4 with int8 operands (worst case `4 × 128 × 128 = 65 536`), but nothing guards it if the array grows.
- **Memories have no reset.** Reads of unwritten addresses return `X`.
- **`valid_in` is one bit broadcast to all rows.** Per-row valid is structurally supported by the PEs but is never driven that way.
- **No `src_mem` → `a_mem`/`weight_mem` load engine.** The host preloads the memories directly through the `host_*` ports. The original `index.md` anticipated this as `agu_load.sv`.
- **No synthesis, timing closure, gate-level simulation, formal verification, or functional coverage.** "13/13 passing" is a pass rate, not a coverage number.

> **VLOG PASSING IS NOT EVIDENCE THE DESIGN BUILDS**
>
> Port-connection type checking happens at **elaboration** (`vopt` / `vsim`), not compilation. A packed/unpacked port mismatch can pass `vlog` cleanly and then produce a pile of `vsim` errors. The regression script elaborates every top, which is what makes its green result meaningful.

Sensible next steps, in order of value

1. **Functional coverage.** Covergroups on phase transitions, `num_vectors` values, and back-to-back versus gapped streaming would turn a pass rate into a coverage number.
2. **Parameter sweep.** Parameterising `systolic_array`'s port widths would let the same testbenches run at 8×8 and 2×2.

3. **Elaboration gating in CI.** Mostly a matter of failing the build on `vopt` errors, which the regression script is already positioned to do.
4. **Overflow handling** on the accumulator, before the array grows.
5. **The load engine** (`agu_load.sv`) so the host is not responsible for preloading.

Companion documents. beginners_guide.html — the same design explained from zero, with line-by-line code walkthroughs. · description_index.md — master file index. · design_files_description.md — per-module reference. · testbench_description.md — per-testbench reference. · verification_report.md — results and coverage boundaries. · context_mkdwn/ — derivations and design decisions.

Regression baseline: 13/13 passing, 3 902 checks, 0 mismatches. QuestaSim 2021.1, run 2026-07-30.